

边上写0。

每个叶子对应一个字符，编码为从根到该叶子的路径上的01序列。在图8-10中，N的编码为001，而E的编码为11。为了证明在一般情况下，都可以用这样的二叉树来表示最优前缀码，需要证明两个结论。

结论1： n 个叶子的二叉树一定对应一个前缀码。如果编码 a 为编码 b 的前缀，则 a 所对应的结点一定为 b 所对应结点的祖先。而两个叶子不会有祖先后代的关系。

结论2：最优前缀码一定可以写成二叉树。逐个字符构造即可。每拿到一个编码，都可以构造出从根到叶子的一条路径，沿着已有结点走，创建不存在的结点。这样得到的二叉树不可能有单子结点，因为如果存在，只要用这个子结点代替父结点，得到的仍然是前缀码，且总长度更短。

接下来的问题就变为：如何构造一棵最优的编码树。

Huffman 算法：把每个字符看作一个单结点子树放在一个树集合中，每棵子树的权值等于相应字符的频率。每次取权值最小的两棵子树合并成一棵新树，并重新放到集合中。新树的权值等于两棵子树权值之和。

下面分两步证明算法的正确性。

结论1：设 x 和 y 是频率最小的两个字符，则存在前缀码使得 x 和 y 具有相同码长，且仅有最后一位编码不同。换句话说，**第一步贪心法选择保留最优解。**

证明：假设深度最大的结点为 a ，则 a 一定有一个兄弟 b 。不妨设 $f(x) \leq f(y)$ ， $f(a) \leq f(b)$ ，则 $f(x) \leq f(a)$ ， $f(y) \leq f(b)$ 。如果 x 不是 a ，则交换 x 和 a ；如果 y 不是 b ，则交换 y 和 b 。这样得到的新编码树不会比原来的差。

结论2：设 T 是加权字符集 C 的最优编码树， x 和 y 是树 T 中两个叶子，且互为兄弟结点， z 是它们的父结点。若把 z 看成具有频率 $f(z)=f(x)+f(y)$ 的字符，则树 $T' = T - \{x, y\}$ 是字符集 $C' = C - \{x, y\} \cup \{z\}$ 的一棵最优编码树。换句话说，**原问题的最优解包含子问题的最优解。**

证明：设 T' 的编码长度为 L ，其中字符 $\{x, y\}$ 的深度为 h ，则把字符 $\{x, y\}$ 拆成两个后，长度变为 $L - (f(x) + f(y)) \cdot h + (f(x) + f(y)) \cdot (h+1) = L + f(x) + f(y)$ 。因此 T' 必须是 C' 的最优编码树， T 才是 C 的最优编码树。

结论1通常称为贪心选择性质，结论2通常称为最优子结构性质。根据这两个结论，Huffman 算法正确。在程序实现上，可以先按照频率把所有字符排序成表 P ，然后创建一个新结点队列 Q ，在每次合并两个结点后把新结点放到队列 Q 中。由于后合并的频率和一定比先合并的频率和大，因此 Q 内的元素是有序的。类似有序表的合并过程，每次只需要检查 P 和 Q 的首元素即可找到频率最小的元素，时间复杂度为 $O(n)$ 。算上排序，总时间复杂度为 $O(n \log n)$ 。

8.5 算法设计与优化策略

本节是本章的重点，也是“基础篇”中第一个贴近竞赛的小节。竞赛中常用的算法设计方法有很多，本节列举一些较为经典的专题，以供读者学习。

构造法。很多时候可以通过“直接构造解”的方法来解决问题。这是最没有规律可循的一种方法，也是最考验“真功夫”的一种方法。

例题 8-1 煎饼 (Stacks of Flapjacks, UVa120)

有一叠煎饼正在锅里。煎饼共有 n ($n \leq 30$) 张，每张都有一个数字，代表它的大小，如图 8-11 所示。厨师每次可以选择一个数 k ，把从锅底开始数第 k 张上面的煎饼全部翻过来，即原来在上面的煎饼现在到了下面。例如，图 8-11 (a)，依次执行操作 3 次后得到图 8-11 (c) 的情况。

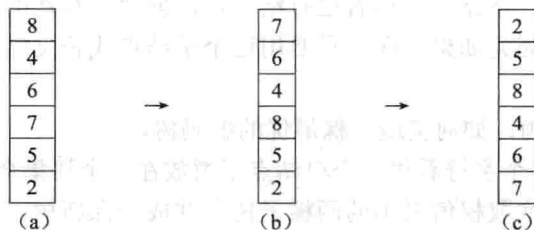


图 8-11 煎饼问题示意图

设计一种方法使得所有煎饼按照从小到大排序（最上面的煎饼最小）。输入时，各个煎饼按照从上到下的顺序给出。例如，上面的例子输入为 8, 4, 6, 7, 5, 2。

【分析】

这道题目要求排序，但是基本操作却是“颠倒一个连续子序列”。不过没有关系，我们还是可以按照选择排序的思想，以从大到小的顺序依次把每个数排到正确的位置。方法是先翻到最上面，然后翻到正确的位置。由于是按照从大到小的顺序处理，当处理第 i 大的煎饼时，是不会影响到第 1, 2, 3, ..., $i-1$ 大的煎饼的（它们已经正确地翻到了煎饼堆底部的 $i-1$ 个位置上）。

例题 8-2 联合国大楼 (Building for UN, ACM/ICPC NEERC 2007, UVa1605)

你的任务是设计一个包含若干层的联合国大楼，其中每层都是一个等大的网格。有若干国家需要在联合国大楼里办公，你需要把每个格子分配给一个国家，使得任意两个不同的国家都有一对相邻的格子（要么是同层中有公共边的格子，要么是相邻层的同一个格子）。你设计的大厦最多不能超过 1000000 个格子。

输入国家的个数 n ($n \leq 50$)，输出大楼的层数 H 、每层楼的行数 W 和列数 L ，然后是每层楼的平面图。不同国家用不同的大小写字母表示。例如， $n=4$ 的一组解是 $H=W=L=2$ ，

第一层是 $\begin{smallmatrix} AB \\ CC \end{smallmatrix}$ ，第二层是 $\begin{smallmatrix} ZZ \\ ZZ \end{smallmatrix}$ 。

【分析】

本题的限制非常少，层数、行数和列数都可以任选。正因为如此，本题的解法非常多。其中有一种方法比较值得探讨：一共只有两层，每层都是 $n \times n$ 的，第一层第 i 行全是国家 i ，第二层第 j 列全是国家 j 。请读者自己验证它是如何满足题目要求的。

中途相遇法。这是一种特殊的算法，大体思路是从两个不同的方向来解决问题，最终“汇集”到一起。第 7 章中提到的“双向广度优先搜索”方法就有一点中途相遇法的味道。

下面再举一个更为直接的例子。

例题 8-3 和为 0 的 4 个值 (4 Values Whose Sum is Zero, ACM/ICPC SWERC 2005, UVa 1152)

给定 4 个 n ($1 \leq n \leq 4000$) 元素集合 A, B, C, D , 要求分别从中选取一个元素 a, b, c, d , 使得 $a+b+c+d=0$ 。问: 有多少种选法?

例如, $A=\{-45, -41, -36, 26, -32\}$, $B=\{22, -27, 53, 30, -38, -54\}$, $C=\{42, 56, -37, -75, -10, -6\}$, $D=\{-16, 30, 77, -46, 62, 45\}$, 则有 5 种选法: $(-45, -27, 42, 30)$, $(26, 30, -10, -46)$, $(-32, 22, 56, -46)$, $(-32, 30, -75, 77)$, $(-32, -54, 56, 30)$ 。

【分析】

最容易想到的算法就是写一个四重循环枚举 a, b, c, d , 看看加起来是否等于 0, 时间复杂度为 $O(n^4)$, 超时。一个稍好的方法是枚举 a, b, c , 则只需要在集合 D 里找找是否有元素 $-a-b-c$, 如果存在, 则方案加 1。如果排序后使用二分查找, 时间复杂度为 $O(n^3 \log n)$ 。

把刚才的方法加以推广, 就可以得到一个更快的算法: 首先枚举 a 和 b , 把所有 $a+b$ 记录下来放在一个有序数组或者 STL 的 map 里, 然后枚举 c 和 d , 查一查 $-c-d$ 有多少种方法写成 $a+b$ 的形式。两个步骤都是 $O(n^2 \log n)$, 总时间复杂度也是 $O(n^2 \log n)$ 。

需要注意的是: 由于本题数据规模较大, 有些时间复杂度为 $O(n^2 \log n)$ 但常数较大的算法在 UVa 上会超时 (例如使用 STL 中的 map 就很容易超时)。笔者推荐的高效实现方法是把所有 $a+b$ 放到一个自己实现的哈希表中, 但建议读者自行尝试不同算法以及实现方法, 这样可以对它们的实际运行效率有一个更直观的认识。

问题分解。有时候可以把一个复杂的问题分解成若干个独立的简单问题, 并加以求解。下面就是一个很好的例子。

例题 8-4 传说中的车 (Fabled Rooks, UVa 11134)

你的任务是在 $n*n$ 的棋盘上放 n ($n \leq 5000$) 个车, 使得任意两个车不相互攻击, 且第 i 个车在一个给定的矩形 R_i 之内。用 4 个整数 xl_i, yl_i, xr_i, yr_i ($1 \leq xl_i \leq xr_i \leq n$, $1 \leq yl_i \leq yr_i \leq n$) 描述第 i 个矩形, 其中 (xl_i, yl_i) 是左上角坐标, (xr_i, yr_i) 是右下角坐标, 则第 i 个车的位置 (x, y) 必须满足 $xl_i \leq x \leq xr_i$, $yl_i \leq y \leq yr_i$ 。如果无解, 输出 IMPOSSIBLE; 否则输出 n 行, 依次为第 1, 2, ..., n 个车的坐标。

【分析】

两个车相互攻击的条件是处于同一行或者同一列, 因此不相互攻击的条件就是不在同一行, 也不在同一列。可以看出: 行和列是无关的, 因此可以把原题分解成两个一维问题。在区间 $[1 \sim n]$ 内选择 n 个不同的整数, 使得第 i 个整数在闭区间 $[n1_i, n2_i]$ 内。是不是很像前面讲过的贪心法题目? 这也是一个不错的练习, 具体解法留给读者思考。

等价转换。与其说这是一种算法设计方法, 还不如说是一种思维方式, 可以帮助选手理清思路, 甚至直接得到问题的解决方案。

例题 8-5 Gergovia 的酒交易 (Wine trading in Gergovia, UVa 11054)

直线上有 n ($2 \leq n \leq 100000$) 个等距的村庄, 每个村庄要么买酒, 要么卖酒。设第 i 个村庄对酒的需求为 a_i ($-1000 \leq a_i \leq 1000$), 其中 $a_i > 0$ 表示买酒, $a_i < 0$ 表示卖酒。所有村庄供需平衡, 即所有 a_i 之和等于 0。

把 k 个单位的酒从一个村庄运到相邻村庄需要 k 个单位的劳动力。计算最少需要多少劳动力可以满足所有村庄的需求。输出保证在 64 位带符号整数的范围内。

【分析】

考虑最左边的村庄。如果需要买酒，即 $a_1 > 0$ ，则一定有劳动力从村庄 2 往左运给村庄 1，而不管这些酒是从哪里来的（可能就是村庄 2 产的，也可能是更右边的村庄运到村庄 2 的）。这样，问题就等价于只有村庄 $2 \sim n$ ，且第 2 个村庄的需求为 $a_1 + a_2$ 的情形。不难发现， $a_1 < 0$ 时这个推理也成立（劳动力同样需要 $|a_1|$ 个单位）。代码如下：

```
int main() {
    int n;
    while(cin >> n && n) {
        long long ans = 0, a, last = 0;
        for(int i = 0; i < n; i++) {
            cin >> a;
            ans += abs(last);
            last += a;
        }
        cout << ans << "\n";
    }
    return 0;
}
```

扫描法。扫描法类似于一种带有顺序的枚举法。例如，从左到右考虑数组的各个元素，也可以说从左到右“扫描”。它和普通枚举法的重要区别是：扫描法往往在枚举时维护一些重要的量，从而简化计算。

例题 8-6 两亲性分子 (Amphiphilic Carbon Molecules, ACM/ICPC Shanghai 2004, UVa1606)

平面上有 n ($n \leq 1000$) 个点，每个点为白点或者黑点。现在需放置一条隔板，使得隔板一侧的白点数加上另一侧的黑点数总数最大。隔板上的点可以看作是在任意一侧。

【分析】

不妨假设隔板一定经过至少两个点（否则可以移动隔板使其经过两个点，并且总数不会变小），则最简单的想法是：枚举两个点，然后输出两侧黑白点的个数。枚举量是 $O(n^2)$ ，再加上统计的 $O(n)$ ，总时间复杂度为 $O(n^3)$ 。

可以先枚举一个基准点，然后将一条直线绕这个点旋转。每当直线扫过一个点，就可以动态修改（这就是“维护”）两侧的点数。在直线旋转“一圈”的过程中，每个点至多被扫描到两次，如图 8-12 所示。因此这个过程的复杂度为 $O(n)$ 。由于扫描之前要将所有点按照相对基准点的极角排序，再加上基准点的 n 种取法，算法的总时间复杂度为 $O(n^2 \log n)$ 。

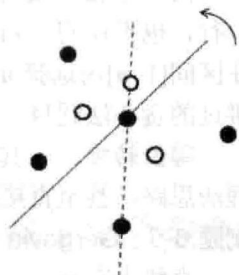


图 8-12 枚举基准点

需要注意的是，本题存在多点共线的情况，如果用反三角函数计算极角，然后判断极角是否相同的话，很容易产生精度误差。

应该把极角相等的条件进行化简（或者直接使用叉积），只使用整数运算进行判断。

滑动窗口。滑动窗口非常有特色，下面的例子很好地说明了这一点。

例题 8-7 唯一的雪花 (Unique snowflakes, UVa 11572)

输入一个长度为 n ($n \leq 10^6$) 的序列 A ，找到一个尽量长的连续子序列 $A_L \sim A_R$ ，使得该序列中没有相同的元素。

【分析】

假设序列元素从 0 开始编号，所求连续子序列的左端点为 L ，右端点为 R 。首先考虑起点 $L=0$ 的情况。可以从 $R=0$ 开始不断增加 R ，相当于把所求序列的右端点往右延伸。当无法延伸（即 $A[R+1]$ 在子序列 $A[L \sim R]$ 中出现过）时，只需增大 L ，并且继续延伸 R 。既然当前的 $A[L \sim R]$ 是可行解， L 增大之后必然还是可行解，所以不必减少 R ，继续增大即可。

不难发现这个算法是正确的，不过真正有意思的是算法的时间复杂度。暂时先不考虑“判断是否可以延伸”这个部分，每次要么把 R 加 1，要么把 L 加 1，而 L 和 R 最多从 0 增加到 $n-1$ ，所以指针增加的次数是 $O(n)$ 的。

最后考虑“判断是否可以延伸”这个部分。比较容易想到的方法是用一个 STL 的 `set`，保存 $A[L \sim R]$ 中元素的集合，当 R 增大时判断 $A[R+1]$ 是否在 `set` 中出现，而 R 加 1 时把 $A[R+1]$ 插入到 `set` 中， $L+1$ 时把 $A[L]$ 从 `set` 中删除。因为 `set` 的插入删除和查找都是 $O(\log n)$ 的，所以这个算法的时间复杂度为 $O(n \log n)$ 。代码如下：

```
#include<cstdio>
#include<set>
#include<algorithm>
using namespace std;

const int maxn = 1000000 + 5;
int A[maxn];

int main() {
    int T, n;
    scanf("%d", &T);
    while(T--) {
        scanf("%d", &n);
        for(int i = 0; i < n; i++) scanf("%d", &A[i]);

        set<int> s;
        int L = 0, R = 0, ans = 0;
        while(R < n) {
            while(R < n && !s.count(A[R])) s.insert(A[R++]);
            ans = max(ans, R - L);
            s.erase(A[L++]);
        }
    }
}
```

```

    printf("%d\n", ans);
}
return 0;
}

```

另一个方法是用一个 `map` 求出 `last[i]`，即下标 i 的“上一个相同元素的下标”。例如，输入序列为 3 2 4 1 3 2 3，当前区间是 $[1, 3]$ （即元素 2, 4, 1），是否可以延伸呢？下一个数是 $A[5]=3$ ，它的“上一个相同位置”是下标 0 ($A[0]=3$)，不在区间中，因此可以延伸。`map` 的所有操作都是 $O(\log n)$ 的，但后面所有操作的时间复杂度均为 $O(1)$ ，总时间复杂度也是 $O(n \log n)$ 。代码如下：

```

#include<cstdio>
#include<map>
using namespace std;

const int maxn = 1000000 + 5;
int A[maxn], last[maxn];
map<int, int> cur;

int main() {
    int T, n;
    scanf("%d", &T);
    while(T--) {
        scanf("%d", &n);
        cur.clear();
        for(int i = 0; i < n; i++) {
            scanf("%d", &A[i]);
            if(!cur.count(A[i])) last[i] = -1;
            else last[i] = cur[A[i]];
            cur[A[i]] = i;
        }

        int L = 0, R = 0, ans = 0;
        while(R < n) {
            while(R < n && last[R] < L) R++;
            ans = max(ans, R - L);
            L++;
        }
        printf("%d\n", ans);
    }
    return 0;
}

```

本题非常经典，请读者仔细品味。

使用数据结构。数据结构往往可以在不改变主算法的前提下提高运行效率，具体做法可能千差万别，但思路却是有规律可循的。下面先介绍一个经典问题。

输入正整数 k 和一个长度为 n 的整数序列 $A_1, A_2, A_3, \dots, A_n$ 。定义 $f(i)$ 表示从元素 i 开始的连续 k 个元素的最小值，即 $f(i) = \min\{A_i, A_{i+1}, \dots, A_{i+k-1}\}$ 。要求计算 $f(1), f(2), f(3), \dots, f(n-k+1)$ 。例如，对于序列 5, 2, 6, 8, 10, 7, 4, $k=4$ ，则 $f(1)=2, f(2)=2, f(3)=6, f(4)=4$ 。

【分析】

如果使用定义，每个 $f(i)$ 都需要 $O(k)$ 时间计算，总时间复杂度为 $O((n-k)k)$ ，太大了。那么换一个思路：计算 $f(1)$ 时，需要求 k 个元素的最小值——这是一个“窗口”。计算 $f(2)$ 时，这个窗口向右滑动了一个位置，计算 $f(3)$ 和 $f(4)$ 时，窗口各滑动了一个位置，如图 8-13 所示。



图 8-13 窗口滑动

因此，这个问题称为**滑动窗口的最小值问题**。窗口在滑动的过程中，窗口中的元素“出去”了一个，又“进来”了一个。借用数据结构中的术语，窗口往右滑动时需要删除一个元素，然后插入一个元素，还需要取最小值。这不就是优先队列吗？第 5 章中曾经介绍过用 STL 集合实现一个支持删除任意元素的优先队列。因为窗口中总是有 k 个元素，插入、删除、取最小值的时间复杂度均为 $O(\log k)$ 。这样，每次把窗口滑动时都需要 $O(\log k)$ 的时间，一共滑动 $n-k$ 次，因此总时间复杂度为 $O((n-k)\log k)$ 。

其实还可以做得更好。假设窗口中有两个元素 1 和 2，且 1 在 2 的右边，会怎样？这意味着 2 在离开窗口之前永远不可能成为最小值。换句话说，这个 2 是无用的，应当及时删除。当删除无用元素之后，滑动窗口中的有用元素从左到右是递增的。为了叙述方便，习惯上称其为**单调队列**。在单调队列中求最小值很容易：队首元素就是最小值。

当窗口滑动时，首先要删除滑动前窗口的最左边元素（如果是有用元素），然后把新元素加入单调队列。注意，比新元素大的元素都变得无用了，应当从右往左删除。如图 8-14 所示是滑动窗口的 4 个位置所对应的单调队列。

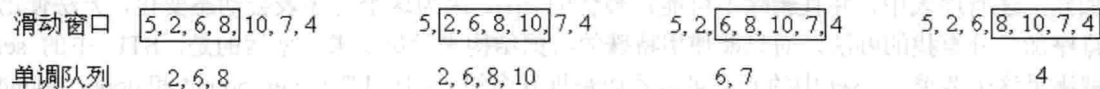


图 8-14 滑动窗口对应的单调队列

单调队列和普通队列有些不同，因为右端既可以插入又可以删除，因此在代码中通常用一个数组和 `front`、`rear` 两个指针来实现，而不是用 STL 中的 `queue`。如果一定要用 STL，则需要用双端队列（即两端都可以插入和删除），即 `deque`。

尽管插入元素时可能会删除多个元素，但因为每个元素最多被删除一次，所以总的时间复杂度仍为 $O(n)$ ，达到了理论下界（因为至少需要 $O(n)$ 的时间来检查每个元素）。

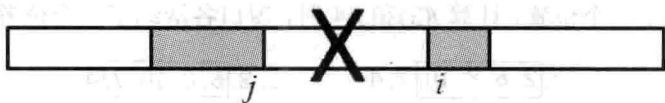
下面这道例题更加复杂，但思路是一样的：先排除一些干扰元素（无用元素），然后把有用的元素组织成易于操作的数据结构。

例题 8-8 防线 (Defense Lines, ACM/ICPC CERC 2010, UVa1471)

给一个长度为 n ($n \leq 200000$) 的序列, 你的任务是删除一个连续子序列, 使得剩下的序列中有一个长度最大的连续递增子序列。例如, 将序列 $\{5, 3, 4, 9, 2, 8, 6, 7, 1\}$ 中的 $\{9, 2, 8\}$ 删除, 得到的序列 $\{5, 3, 4, 6, 7, 1\}$ 中包含一个长度为 4 的连续递增子序列 $\{3, 4, 6, 7\}$ 。序列中每个数均为不超过 10^9 的正整数。

【分析】

为了方便叙述, 下面用 L 序列表示“连续递增子序列”。删除一个子序列之后, 得到的最长 L 序列应该是由两个序列拼起来的, 如图 8-15 所示。

图 8-15 最长序列 L

最容易想到的算法是枚举 j 和 i (前提是 $A[j] < A[i]$, 否则拼不起来), 然后分别往左和往右数一数最远能延伸到哪里。枚举量为 $O(n^2)$, 而“数一数”的时间复杂度为 $O(n)$, 因此总时间复杂度为 $O(n^3)$ 。

加上一个预处理, 就能避免“数一数”这个过程, 从而把时间复杂度降为 $O(n^2)$ 。设 $f(i)$ 为以第 i 个元素开头的最长 L 序列长度, $g(i)$ 为以第 i 个元素结尾的最长 L 序列长度, 则不难在 $O(n)$ 时间内求出 $f(i)$ 和 $g(i)$, 然后枚举完 j 和 i 之后, 最长 L 序列的长度就是 $g(j) + f(i)$ 。

还可以做得更好: 只枚举 i , 不枚举 j , 而是用其他方法快速找一个 $j < i$, 使得 $A[j] < A[i]$, 且 $g(j)$ 尽量大。如何快速找到呢? 首先要排除一些肯定不是最优值的 j 。例如, 若有 j' 满足 $A[j'] \leq A[j]$ 且 $g(j') > g(j)$, 则 j 肯定不满足条件, 因为 j' 不仅是一个更长的 L 序列的末尾, 而且它更容易拼成。

这样, 把所有“有保留价值”的 j 按照 $A[j]$ 从小到大排成一个有序表 (根据刚才的结论, $A[j]$ 相同的 j 只保留一个), 则 g 也会是从小到大排列。那么用二分查找找到满足 $A[j] < A[i]$ 的最大的 $A[j]$, 则它对应的 $g(j)$ 也是最大的。

不过这个方法只有当 i 固定时才有效。实际上每次计算完一个 $g(i)$ 之后, 还要把这个 $A[i]$ 加到上述有序表中, 并且删除不可能是最优的 $A[j]$ 。因为这个有序表会动态变化, 无法使用排序加二分查找的办法, 而只能使用特殊的数据结构来满足要求。幸运的是, STL 中的 `set` 就满足这个要求——`set` 中的元素可以看成是排好序的, 而且自带 `lower_bound` 和 `upper_bound` 函数, 作用和之前讨论过的一样。

为了方便起见, 此处用二元组 $(A[j], g(j))$ 表示这些“有保留价值”的东西, 如 $(10, 4)$, $(20, 8)$, $(30, 15)$, $(40, 18)$, $(50, 30)$, 并且以 $A[j]$ 为关键字放在一个 STL 集合中。对于固定的 i , 不难用 `lower_bound` 满足 $A[j] < A[i]$ 的最大 $A[j]$, 以及对应的 $g(j)$, 真正复杂的是这个集合本身的更新, 即前面提到的“每次计算完一个 $g(i)$ 之后”需要做的事情。

假设已经计算出一个 $g(i) = 6$, 且 $A[i] = 25$, 接下来会发生什么事情? 首先把 $(25, 6)$ 插入集合中, 然后检查它的前一个元素 $(20, 8)$ 。由于 $20 < 25$, $8 > 6$, $(25, 6)$ 是不应该保留的。但如果插入的是 $(25, 20)$, 情况就完全不同了: 不仅 $(25, 20)$ 需要保留, 而且还要删除 $(30, 15)$ 和 $(40, 18)$ 。一般地, 插入任何一个二元组时首先应找到其插入位置, 根据它前一个元素判断是否需要

保留。如果需要保留,再往后遍历,删除所有不再需要保留的元素。因为所有元素至多被删除一次,而查找、插入和删除的时间复杂度均为 $O(\log n)$,所以消耗在 STL 集合上的总时间复杂度为 $O(n \log n)$ ^①。

数形结合。数形结合是一种相对高级的算法设计策略,虽有一定规律可循,但仍然灵活多变。通过下面的例题,读者可对其中的奥妙了解一二。

例题 8-9 平均值 (Average, Seoul 2009, UVa1451)

给定一个长度为 n 的 01 串,选一个长度至少为 L 的连续子串,使得子串中数字的平均值最大。如果有多解,子串长度应尽量小;如果仍有多解,起点编号尽量小。序列中的字符编号为 $1 \sim n$,因此 $[1, n]$ 就是完整的字符串。 $1 \leq n \leq 100000$, $1 \leq L \leq 1000$ 。

例如,对于如下长度为 17 的序列 0010101101101010,如果 $L=7$,最大平均值为 $6/8$ (子序列为 $[7, 14]$,其长度为 8);如果 $L=5$,子序列 $[7, 11]$ 的平均值最大,为 $4/5$ 。

【分析】

先求前缀和 $S_i = A_1 + A_2 + \dots + A_i$ (规定 $S_0 = 0$),然后令点 $P_i = (i, S_i)$,则子序列 $i \sim j$ 的平均值为 $(S_j - S_{i-1}) / (j - i + 1)$,也就是直线 $P_{i-1}P_j$ 的斜率。这样可得到主算法:从小到大枚举 t ,快速找到 $t' \leq t - L$,使得 $P_{t'}P_t$ 斜率最大。注意题目中的 A_i 都是 0 或 1,因此每个 P_i 和上一个 P_{i-1} 相比,都是 x 加 1, y 不变或者加 1。

对于给定的 t ,要找的点 $P_{t'}$ 在 P_t 的左边。假设有 3 个候选点 P_i, P_j, P_k ,下标满足 $i < j < k < t$,并且 3 个点成上凸形状 (P_j 为上凸点)。假设 P_t 的 x 坐标为 x_0 ,根据定义, P_t 的 y 坐标一定不小于 P_k 的 y 坐标,因此 P_t 一定位于 A, B, C 3 条线段/射线之一,如图 8-16 所示。

□ 当 P_t 在射线 A 上时, P_k 比 P_j 好 (即 P_kP_t 的斜率比 P_jP_t 的斜率大,后同)。

□ 当 P_t 在线段 B 上时, P_i 比 P_j 好。

□ 当 P_t 在线段 C 上时, P_i 和 P_k 都比 P_j 好。

换句话说,只要出现上凸的情况,上凸点一定可以忽略。

假设已经有了一些下凸点,现在又加入了一个点,可能会使一些已有的点变为上凸点,这时就应当将这些上凸点删除。由于被删除的点总是原来的下凸点中最右边的若干个连续点,所以可以用栈来实现,如图 8-17 所示。

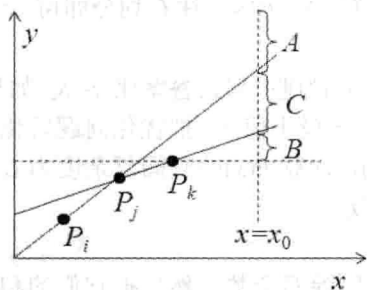


图 8-16 平均值问题示意图

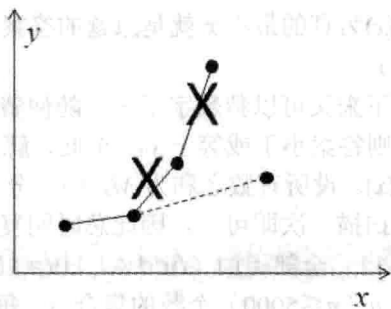


图 8-17 下凸点

^① 有一个办法可以避开 STL 集合: 用一个数组保存每个 $g(j)$ 对应的最小的 $A[j]$ 。细节请参考《算法竞赛入门经典——训练指南》第 1 章中介绍的 LIS 的 $O(n \log n)$ 算法。